Hashing

Introduction to **Hashing**

- **Hashing** is a technique used to store and retrieve data quickly by converting a key into an index (address) of an array using a **hash function**.
- Instead of searching through all elements one by one, hashing allows us to access data in **O(1) average time complexity**.
- **Basic Idea**
- A **hash function** takes a key and generates an index where the data will be stored.

Example

- Suppose we have an array of size 10 and a hash function:
- $h(k) = k \bmod 10$
- For key **25**:
- 25 mod 10 = 5
- So, the value associated with key 25 will be stored at index **5**.

Key	Hash Value (Index)
25	5
37	7
42	2
58	8

Components of Hashing

- **1. Key :** The data value used for searching.
- Example:
  - Roll Number
  - Employee ID
  - Username
- **2. Hash Function :** A function that converts a key into an array index.
- Example:
  - h(key) = key % table_size
- **3. Hash Table :** A data structure that stores key-value pairs.

Index	Value
0	-
1	-
2	42
3	-
4	-
5	25
6	-
7	37
8	58
9	-

Operations in Hashing

- **Insertion**
  - Store data at the index generated by the hash function.
- **Searching**
  - Calculate the hash value and directly access that index.
- **Deletion**
  - Find the element using hashing and remove it.

Hash Collision

- A **collision** occurs when two different keys generate the same hash value.
- **Example**

  Using:

  h(k) = k % 10

  25 % 10 = 5

  35 % 10 = 5
- Both keys map to index 5.
- This creates a collision.

Collision Resolution Techniques

- **1. Separate Chaining**

Store multiple elements at the same index using a linked list.

Index 5

25 -> 35 -> 45

- **2. Linear Probing**

Find the next empty location.

25 -> Index 5
35 -> Index 6
45 -> Index 7

- **3. Quadratic Probing**

  Check positions using square increments.

  $h(k)+1^2$

  $h(k)+2^2$

  $h(k)+3^2$

- **4. Double Hashing**

  Use a second hash function to find the next position.

Applications of Hashing

- Dictionaries
- Maps
- Sets
- Database Indexing
- Caching Systems
- Password Storage
- Symbol Tables in Compilers
- Search Engines

Time Complexity

Operation	Average Case	Worst Case
Search	O(1)	O(n)
Insert	O(1)	O(n)
Delete	O(1)	O(n)

Hashing Implementation

- 1. Simple Hash Table Implementation
- This example uses:
  - Array as Hash Table
  - Hash Function: key % SIZE
  - Integer keys only
  - No collision handling

```
#include <stdio.h>

#define SIZE 10

int hashTable[SIZE];

int hashFunction(int key)
{
   return key % SIZE;
}

void insert(int key)
{
   int index = hashFunction(key);
   hashTable[index] = key;
}
```

```
void search(int key)
{
int index = hashFunction(key);

if(hashTable[index] == key)
printf("Key Found at index
%d\n", index);
else
printf("Key Not Found\n");
}
```

```
void display()
{
   int i;

   printf("\nHash Table:\n");

   for(i=0;i<SIZE;i++)
      printf("%d --> %d\n", i,
hashTable[i]);
}
```

```
int main()
{
   insert(25);
   insert(37);
   insert(42);
   insert(58);

   display();

   search(37);
   search(50);

   return 0;
}
```

2. Hashing with Linear Probing (Collision Handling)

- When a collision occurs, the next empty location is searched.
- **Example**
- Hash Function: h(k) = k % 10

25 → 5
35 → 5 (Collision)

Store 35 at index 6

```
#include <stdio.h>

#define SIZE 10

int hashTable[SIZE];

void initialize()
{
int i;
for(i=0;i<SIZE;i++)
hashTable[i] = -1;
}

int hashFunction(int key)
{
return key % SIZE;
}
```

```
void insert(int key)
{
int index = hashFunction(key);

while(hashTable[index] != -1)
{
index = (index + 1) % SIZE;
}

hashTable[index] = key;
}
```

```
void search(int key)
{
int index = hashFunction(key);
int start = index;

while(hashTable[index] != -1)
{
if(hashTable[index] == key)
{
printf("Key Found at index
%d\n", index);
return;
}

index = (index + 1) % SIZE;

if(index == start)
break;
}

printf("Key Not Found\n");
}
```

```
void display()
{
int i;

printf("\nHash Table:\n");

for(i=0;i<SIZE;i++)
{
if(hashTable[i] == -1)
printf("%d --> Empty\n",
i);
else
printf("%d --> %d\n", i,
hashTable[i]);
}
}
```

```
int main()
{
initialize();

insert(25);
insert(35);
insert(45);
insert(56);

display();

search(35);
search(99);

return 0;
}
```

Hashing Implementation in Python

```
SIZE = 10
hash_table = [None] * SIZE

def hash_function(key):
    return key % SIZE

def insert(key):
    index = hash_function(key)

    while hash_table[index] is not None:
        index = (index + 1) % SIZE

    hash_table[index] = key

def display():
    for i in range(SIZE):
        print(i, "->", hash_table[i])
```

```
insert(25)
insert(35)
insert(45)
insert(56)

display()
```